

20.图书系统项目（四）

本节当中主要完成业务逻辑框架的搭建，让整个项目的框架可以跑起来！

1. ProxyUser类完善业务逻辑框架

ProxyUser 类添加如下方法：

- 每个业务需要判断该代理对象是否是有权限的
- 调用对应用户当中的方法进行操作
- 注意：查找图书，显示图书，退出系统是2个角色都有的方法，这些方法可以提出到Library当中，不需要区分

```
1 private void checkRealUserWhetherAdminUser(String exceptionMessage) {
2     if(!(realUser instanceof AdminUser)) {
3         //自定义异常，自己定义即可
4         throw new PermissionException(exceptionMessage);
5     }
6 }
7
8 public void addBook() {
9     //查看代理的对象是不是管理员
10    checkRealUserWhetherAdminUser("普通用户没有权限上架图书");
11    ((AdminUser) realUser).addBook();
12 }
13
14 //更新书籍操作
15 public void updateBook() {
16 }
17
18 //移除图书
19 public void removeBook() {
20 }
21
22 //查看图书的借阅次数
23 public void borrowCount( ) {
24 }
25
26 //查看最受欢迎的前K本书
27 public void generateBook() {
28 }
```

```

29
30 //查看库存状态
31 public void checkInventoryStatus() {
32 }
33
34 //移除上架超过1年的书籍
35 public void checkAndRemoveOldBooks() {
36 }
37
38 //-----普通相关方法-----//
39 private void checkRealUserWhetherNormalUser(String exceptionMessage) {
40     if(!(realUser instanceof NormalUser)){
41         throw new PermissionException(exceptionMessage);
42     }
43 }
44 //借阅图书
45 public void borrowBook() {
46 }
47
48 //归还图书
49 public void returnBook() {
50 }
51
52
53 //查看个人借阅情况
54 public void viewBorrowHistory() {
55 }

```

`handleOperation` 方法，根据选择进行对应操作

查找图书、显示图书、退出系统三种操作对于管理员和普通用户来说都是共用的，所以直接定义在 Library 当中

```

1 public void handleOperation(int choice) {
2     if (realUser instanceof AdminUser) {
3         // 管理员操作
4         switch (choice) {
5             case Constant.SEARCH_BOOK:
6                 library.searchBook();
7                 break;
8             case Constant.DISPLAY_BOOK:
9                 library.displayBooks();
10                break;
11             case Constant.EXIT:
12                library.exit();

```

```

13         break;
14     case Constant.ADD_BOOK:
15         addBook();
16         break;
17     case Constant.UPDATE_BOOK:
18         updateBook();
19         break;
20     case Constant.REMOVE_BOOK:
21         removeBook();
22         break;
23     case Constant.BORROWED_BOOK_COUNT:
24         borrowCount();
25         break;
26     case Constant.GENERATE_BOOK:
27         generateBook();
28         break;
29     case Constant.CHECK_INVENTORY_STATUS:
30         checkInventoryStatus();
31         break;
32     case Constant.CHECK_AND_REMOVE_OLD_BOOK:
33         checkAndRemoveOldBooks();
34         break;
35     default:
36         System.out.println("无效的操作。");
37     }
38 }else if (realUser instanceof NormalUser) {
39     // 普通用户操作
40     switch (choice) {
41         case Constant.SEARCH_BOOK:
42             library.searchBook();
43             break;
44         case Constant.DISPLAY_BOOK:
45             library.displayBooks();
46             break;
47         case Constant.EXIT:
48             library.exit();
49         case Constant.BORROWED_BOOK:
50             borrowBook();
51             break;
52         case Constant.RETURN_BOOK:
53             returnBook();
54             break;
55         case Constant.VIEW_BORROW_HISTORY_BOOK:
56             viewBorrowHistory();
57             break;
58         default:
59             System.out.println("无效的操作。");

```

```

60     }
61 }
62 }

```

- 这里并没有使用1 2 3 等常数作为case的判断条件，而是使用了常量来表示
- 1 2 3 这种也被称为 magic number（幻数），缺点在于如果只是使用数字，别的程序员可能并不能读懂甚至过一段时间，自己也可能看不懂。所以定义为常量可以增加代码的可读性。

继续完善 Constant 类：

```

1  public class Constant {
2
3      //内存中的书籍数组初识容量
4      public static final int CAPACITY = 5;
5
6      //存储所有图书的文件
7      public static final String ALL_BOOK_FILE_NAME = "allbook.txt";
8
9
10     //-----管理员相关操作管理-----
11     //查找图书
12     public static final int SEARCH_BOOK = 1;
13     //显示图书
14     public static final int DISPLAY_BOOK = 2;
15     //退出系统
16     public static final int EXIT = 3;
17     //上架图书
18     public static final int ADD_BOOK = 4;
19     //更新图书
20     public static final int UPDATE_BOOK = 5;
21     //删除图书
22     public static final int REMOVE_BOOK = 6;
23     //查看图书的借阅次数
24     public static final int BORROWED_BOOK_COUNT = 7;
25     //查看受欢迎的图书
26     public static final int GENERATE_BOOK = 8;
27     //查看库存状态
28     public static final int CHECK_INVENTORY_STATUS = 9;
29     //移除上架超过1年的书籍
30     public static final int CHECK_AND_REMOVE_OLD_BOOK = 10;
31
32     //-----普通用户相关操作管理-----
33
34     //借阅图书
35     public static final int BORROWED_BOOK = 4;

```

```

36
37     //归还图书
38     public static final int RETURN_BOOK = 5;
39
40     //查看个人借阅情况
41     public static final int VIEW_BORROW_HISTORY_BOOK = 6;
42
43 }

```

总结：

1. 完善了代理类中方法的定义和整合，根据 `handleOperation` 方法进行实际选择
2. ProxyUser类代理了 AdminUser类和NormalUser类中的方法

2. AdminUser 类完善业务逻辑框架

由上节课可知，该类当中需要做一些准备工作，比如添加图书的时候需要做数据准备工作！

```

1  public class AdminUser extends User{
2
3      private Scanner scanner = null;
4
5      public AdminUser(String name,int userID) {
6          super(name,userID,"管理员");
7          scanner = new Scanner(System.in);
8      }
9      @Override
10     public int display() {
11         System.out.println("管理员 " + name + " 的操作菜单:");
12         System.out.println("1. 查找图书");
13         System.out.println("2. 打印所有的图书");
14         System.out.println("3. 退出系统");
15         System.out.println("4. 上架图书");
16         System.out.println("5. 修改图书");
17         System.out.println("6. 下架图书");
18         System.out.println("7. 统计借阅次数");
19         System.out.println("8. 查看最后欢迎的前K本书");
20         System.out.println("9. 查看库存状态");
21         System.out.println("10. 检查超过一年未下架的图书");
22         System.out.println("请选择你的操作: ");
23         return scanner.nextInt();
24     }
25     //其他操作方法
26     //上架图书

```

```
27     public void addBook() {
28         scanner.nextLine();
29         System.out.println("请输入书名: ");
30         String title = scanner.nextLine(); // 输入书名
31         System.out.println("请输入作者: ");
32         String author = scanner.nextLine(); // 输入作者
33         System.out.println("请输入类别: ");
34         String category = scanner.nextLine(); // 输入图书类别
35         System.out.println("请输入出版年份: ");
36         int year = scanner.nextInt(); // 输入出版年份
37         scanner.nextLine(); // 吞掉换行符
38
39         LocalDate shelfDate = LocalDate.now(); // 当前时间作为上架时间
40         Book newBook = new Book(title, author, category, year, shelfDate);
41         // 创建新书对象
42
43         //调用图书类 添加图书
44         library.addBook(newBook);
45     }
46
47     //图书修改 支持修改书名 作者 类别
48     public void updateBook() {
49
50     }
51
52     //删除书籍
53     public void removeBook() {
54
55     }
56
57     //统计每本书的借阅次数
58     public void borrowCount() {
59         //不需要准备其他工作，直接调用具体的业务实现
60         library.borrowCount();
61     }
62
63     //查询最受欢迎的前n本书 【借阅报告】
64     public void generateBook() {
65
66     }
67
68     //查看库存状态
69     public void checkInventoryStatus() {
70
71     }
72
73     //并移除上架超过一年的图书
```

```

73     public void checkAndRemoveOldBooks() {
74
75     }
76
77     public void exit() {
78         library.exit();
79     }
80
81 }

```

- 上述方法中有的需要做数据准备，比如添加图书等，但是有些不需要比如**统计每本书的借阅次数**等。我们就直接调用 `Library` 中的方法即可
- 我们观察到对于 `Scanner` 对象来说，不仅仅在 `AdminUser` 当中会使用到，而且将来也可能在其他地方使用到，此时我们可以把 `Scanner` 这个类设计为单例模式。保证整个项目只有这一个实例，大家都可以使用。

2.1 单例模式设计 `Scanner` 类和 `Library` 类

在utils下创建2个类： `ScannerSingleton` 和 `LibrarySingleton`

2.1.1 单例模式设计 `Scanner`

```

1  package utils;
2
3  import java.util.Scanner;
4
5  public class ScannerSingleton {
6      private static Scanner scanner;
7
8      private ScannerSingleton() {
9
10     }
11     public static Scanner getInstance() {
12         if (scanner == null) {
13             scanner = new Scanner(System.in);
14         }
15         return scanner;
16     }
17
18 }

```

2.1.2 单例模式设计 `Library`

```

1  package utils;
2
3  import book.Library;
4
5  public class LibrarySingleton {
6      private static Library library;
7
8      private LibrarySingleton() {
9
10     }
11     public static Library getInstance() {
12         if (library == null) {
13             library = new Library();
14         }
15         return library;
16     }
17 }

```

我们在使用到 `Scanner` 和 `Library` 的地方，进行更改

```

1  public class AdminUser extends User{
2      private Scanner scanner = null;
3      private Library library = null;
4
5      public AdminUser(String name,int userID) {
6          super(name,userID,"管理员");
7          library = LibrarySingleton.getInstance();
8          scanner = ScannerSingleton.getInstance();
9      }
10 }

```

```

1  public class ProxyUser {
2      //被代理的真实用户
3      private User realUser;
4      private Library library = null;
5
6      public ProxyUser(User user) {
7          //对被代理的对象进行赋值
8          realUser = user;
9          library = LibrarySingleton.getInstance();

```

```
10     }  
11 }
```

3. NormalUser 类完善业务逻辑框架

在设计该类的时候，我们需要想清楚一个逻辑。

- 用户借书怎么设计？
- 多个用户借书怎么设计？
- 借书之后存储到文件当中的格式是什么？

3.1 PairOfUidAndBookId 类设计

我们的设计是这样的：

- 文件当中存储借阅书籍信息为：用户ID,书籍ID的形式，每一行是一个借阅信息

1,101 :表示ID为1的用户借阅了ID为101的书籍

2,111

1,301

所以我们就涉及到了读取文件后，内存当中需要存储这些借阅信息。我们定义类

`PairOfUidAndBookId` 来表示一条借阅信息，多条信息使用数组即可。

在book包当中定义

```
1  public class PairOfUidAndBookId {  
2      private int userId;  
3  
4      private int bookId;  
5  
6      public PairOfUidAndBookId() {  
7  
8      }  
9  
10     public PairOfUidAndBookId(int userId, int bookId) {  
11         this.userId = userId;  
12         this.bookId = bookId;  
13     }  
14  
15     public int getUserId() {  
16         return userId;  
17     }  
18 }
```

```

19     public void setUserId(int userId) {
20         this.userId = userId;
21     }
22
23     public int getBookId() {
24         return bookId;
25     }
26
27     public void setBookId(int bookId) {
28         this.bookId = bookId;
29     }
30     //把对象序列化为JSON字符串的形式
31     public String toJson() {
32         StringBuilder json = new StringBuilder();
33         json.append(userId).append(",");
34         json.append(bookId);
35         return json.toString();
36     }
37 }

```

3.2 AnalyzingBorrowedBook 类的设计

util包中定义

该设计方案和 AnalyzingBook 设计是类似的

```

1  public class AnalyzingBorrowedBook {
2
3
4      public void storeObject(PairOfUidAndBookId[] pairOfUidAndBookIds, String
filename) throws IOException {
5
6          //先遍历pairOfUidAndBookIds数组当中不为空的数据多少个?
7          int booksUseLen = 0;
8          for (int i = 0; i < pairOfUidAndBookIds.length; i++) {
9              if(pairOfUidAndBookIds[i] != null) {
10                  booksUseLen++;
11              }
12          }
13
14          StringBuilder jsonArray = new StringBuilder();
15
16          for (int i = 0; i < booksUseLen; i++) {
17              if(pairOfUidAndBookIds[i] != null) {
18                  jsonArray.append(pairOfUidAndBookIds[i].toJson());

```

```

19         if (i != booksUseLen-1) {
20             jsonArray.append("\n");
21         }
22     }
23 }
24
25 FileUtils.writeFile(jsonArray.toString(),filename);/* */
26 }
27
28
29 public PairOfUidAndBookId[] loadObject(String filename) throws
IOException {
30     //从文件读取数据
31     String content = FileUtils.readFile(filename);
32     if (content == null || content.isEmpty()) {
33         System.out.println("已借阅书籍列表无数据，表示没有用户借阅过书籍");
34         return null;
35     }
36
37     String[] JsonStrings = content.split("\n");
38
39     PairOfUidAndBookId[] pairOfUidAndBookIds
40         = new
PairOfUidAndBookId[JsonStrings.length];
41
42
43     for (int i = 0; i < JsonStrings.length; i++) {
44         PairOfUidAndBookId pairOfUidAndBookId = new PairOfUidAndBookId();
45         String[] uidAndBookIds = JsonStrings[i].split(",");
46         pairOfUidAndBookId.setUserId(Integer.parseInt(uidAndBookIds[0]));
47         pairOfUidAndBookId.setBookId(Integer.parseInt(uidAndBookIds[1]));
48
49         pairOfUidAndBookIds[i] = pairOfUidAndBookId;
50     }
51
52     return pairOfUidAndBookIds;
53 }
54
55
56 }

```

3.3 NormalUser 类完善

```

1 public class NormalUser extends User {
2

```

```

3      // 用户已借阅的图书相关信息
4      private PairOfUidAndBookId[] pairOfUidAndBookIds;
5      // 当前书籍的借阅量
6      private int borrowedCount;
7      //最多借阅的图书数量
8      private static final int BORROW_BOOK_MAX_NUM = 5;
9      private final AnalyzingBorrowedBook analyzingBorrowedBook = new
AnalyzingBorrowedBook();
10
11
12      private Scanner scanner = null;
13      private Library library = null;
14
15      //如果是普通用户，这里写死
16      public NormalUser(String name, int userID) {
17          super(name, userID, "普通用户");
18          loadBorrowedBook();
19          library = LibrarySingleton.getInstance();
20          scanner = ScannerSingleton.getInstance();
21      }
22
23      private void loadBorrowedBook() {
24          PairOfUidAndBookId[] allBorrowedBook;
25          try {
26              //1.先加载文件当中的借阅信息
27              allBorrowedBook=
analyzingBorrowedBook.loadObject(Constant.BORROWED_BOOK_FILE_NAME);
28              //2. 默认已借阅的图书数组大小为BORROW_BOOK_MAX_NUM，这里也可以定义到常量
类
29              pairOfUidAndBookIds = new PairOfUidAndBookId[BORROW_BOOK_MAX_NUM];
30              //3.没有读取到已借阅的图书信息
31              if (allBorrowedBook== null) {
32                  borrowedCount = 0;
33              } else {
34                  //4. 查看实际读取到的数组长度是多少？
35                  int allBorrowedBookLen= allBorrowedBook.length;
36                  //5. 如果读取到了10本书被借阅 但是当前borrowedBooks数组长度小于10
37                  if (allBorrowedBookLen> pairOfUidAndBookIds.length) {
38                      //6. 按照实际情况进行分配数组内存
39                      pairOfUidAndBookIds = new
PairOfUidAndBookId[allBorrowedBookLen];
40                  }
41                  //7.把数据拷贝回到 已借阅图书信息的数组当中
42                  for (int i = 0; i < allBorrowedBookLen; i++) {
43                      pairOfUidAndBookIds[i] = allBorrowedBook[i];
44                  }
45                  //8.更新当前实际借阅书籍的书籍数量

```

```
46         borrowedCount = allBorrowedBookLen;
47     }
48     } catch (IOException e) {
49         throw new RuntimeException(e);
50     }
51
52 }
53
54 private void storeBorrowedBook() {
55     try {
56         analyzingBorrowedBook.storeObject(pairOfUidAndBookIds,
Constant.BORROWED_BOOK_FILE_NAME);
57     } catch (IOException e) {
58         throw new RuntimeException(e);
59     }
60 }
61
62 @Override
63 public int display() {
64     System.out.println("普通用户 " + name + " 的操作菜单:");
65     System.out.println("1. 查找图书");
66     System.out.println("2. 打印所有的图书");
67     System.out.println("3. 退出系统");
68     System.out.println("4. 借阅图书");
69     System.out.println("5. 归还图书");
70     System.out.println("6. 查看当前个人借阅情况");
71     System.out.println("请选择你的操作: ");
72     return scanner.nextInt();
73 }
74
75 //借阅图书
76 public void borrowBook() {
77
78 }
79
80
81 //归还图书
82 public void returnBook() {
83
84 }
85
86 // 查看个人借阅情况
87 public void viewBorrowBooks() {
88
89 }
90
91 }
```

常量类添加路径：

▼ 代码块

```
1 public static final String BORROWED_BOOK_FILE_NAME = "borrowedbook.txt";
```

完

比特就业课